

Императрица - Gramática Completa e Exemplos

Tabela de Conteúdos

1. Gramática BNF
2. Tokens e Expressões Regulares
3. Exemplos Práticos

Gramática BNF

Notação

- Termos em **CAPS LOCK** são terminais (tokens)

Produção Principal

file $\rightarrow$ bloco EOF

bloco $\rightarrow$ cmd bloco | ϵ

cmd $\rightarrow$ cmdIf | cmdFor | cmdWhile | cmdReturn | cmdDefFunc | cmdPrint | cmdInput | cmdID | OP_CONTINUE | OP_BREAK

Comandos de Controle de Fluxo

cmdIf $\rightarrow$ OP_IF valor OPEN_BRACES bloco CLOSE_BRACES cmdElse

cmdElse $\rightarrow$ OP_ELSE OPEN_BRACES bloco CLOSE_BRACES | ϵ

cmdWhile $\rightarrow$ OP_WHILE valor OPEN_BRACES bloco CLOSE_BRACES

cmdFor $\rightarrow$ OP_FOR OPEN_PARENTHESIS variavelFor SEMICOLON expressaoRelacional SEMICOLON expressaoAritmetica
CLOSE_PARENTHESIS OPEN_BRACES bloco CLOSE_BRACES

variavelFor $\rightarrow$ tipo ID complemento | ID complemento

Definição de Funções e Retorno

cmdDefFunc $\rightarrow$ OP_FUNCTION ID OPEN_PARENTHESIS listaParametros CLOSE_PARENTHESIS OPEN_BRACES bloco CLOSE_BRACES

listaParametros $\rightarrow$ ID entradaListaParam | ϵ

entradaListaParam $\rightarrow$ COMMA ID entradaListaParam | ϵ

cmdReturn $\rightarrow$ OP_RETURN valorRetorno

valorRetorno $\rightarrow$ valor | ϵ

Declaração e Atribuição de Variáveis

cmdID $\rightarrow$ tipo ID acessoListaOp complemento SEMICOLON
| ID acessoListaOp complemento SEMICOLON

acessoListaOp $\rightarrow$ acessoLista acessoListaOp | ϵ

acessoLista $\rightarrow$ OPEN_BRACKETS expressaoAritmetica CLOSE_BRACKETS

complemento $\rightarrow$ ASSIGN valor
| operadorAssignOp valor
| OPEN_PARENTHESIS corpoLista CLOSE_PARENTHESIS
| ϵ

operadorAssignOp $\rightarrow$ PLUS_ASSIGN | MINUS_ASSIGN | TIMES_ASSIGN | DIV_ASSIGN | MOD_ASSIGN | POW_ASSIGN

Entrada/Saída

cmdPrint → OP_PRINT OPEN_PARENTHESIS corpoLista CLOSE_PARENTHESIS SEMICOLON

cmdInput → OP_INPUT OPEN_PARENTHESIS corpoLista CLOSE_PARENTHESIS SEMICOLON

Expressões

valor → expressaoLogica | lista | cmdPrint | cmdInput

expressaoLogica → expressaoRelacional (AND expressaoRelacional)*
| expressaoRelacional (OR expressaoRelacional)*

expressaoRelacional → expressaoAritmetica (opComparacao expressaoAritmetica)*

opComparacao → GREATER | LESS | EQUAL | DIFFERENT | GREATER_EQUAL | LESS_EQUAL

expressaoAritmetica → termo (opAdicao termo)*

opAdicao → PLUS | MINUS

termo → fator (opMultiplicacao fator)*

opMultiplicacao → TIMES | DIV | MOD

fator → elemento (POW elemento)*

elemento → INCREMENT ID X
| DECREMENT ID X
| ID X
| INT
| FLOAT
| STR
| BOOL
| OPEN_PARENTHESIS expressaoLogica CLOSE_PARENTHESIS

X → composicao X | INCREMENT | DECREMENT | ε

composicao → OPEN_BRACKETS expressaoAritmetica CLOSE_BRACKETS acessoListaOp
| OPEN_PARENTHESIS corpoLista CLOSE_PARENTHESIS

corpoLista → valor entradaLista | ε

entradaLista → COMMA valor entradaLista | ε

lista → OPEN_BRACKETS corpoLista CLOSE_BRACKETS

tipo → INT_TYPE | FLOAT_TYPE | BOOL_TYPE | STR_TYPE

Tokens e Expressões Regulares

Tipos de Dados

Palavra-chave	Tipo
интеграл	Inteiro (int)
десятичный	Decimal (float)
строка	String (char*)
логический	Booleano (bool)

Valores Booleanos

Palavra-chave	Valor
истинный	Verdadeiro (true)
ложь	Falso (false)

Operadores de Controle

Palavra-chave	Significado
если	If (se)
иначе	Else (senão)
пока	While (enquanto)
для	For (para)
функция	Function (função)
вернуть	Return (retornar)
печать	Print (imprimir)
входной	Input (entrada)
продолжать	Continue (continuar)
перерыв	Break (quebra)
делать	Do (fazer)

Operadores Aritméticos

Token	Símbolo	Significado
PLUS	+	Adição
MINUS	-	Subtração
TIMES	*	Multiplificação
DIV	/	Divisão
MOD	%	Módulo (resto)
POW	^	Potência
INCREMENT	++	Incremento
DECREMENT	--	Decremento

Operadores de Atribuição

Token	Símbolo
ASSIGN	:=
PLUS_ASSIGN	+=
MINUS_ASSIGN	-=
TIMES_ASSIGN	*=
POW_ASSIGN	^=

Operadores Relacionais

Token	Símbolo	Significado
GREATER	>	Maior que
LESS	<	Menor que
GREATER_EQUAL	>=	Maior ou igual
LESS_EQUAL	<=	Menor ou igual
EQUAL	==	Igual
DIFFERENT	!=	Diferente

Operadores Lógicos

Token	Símbolo	Significado
AND	&	E lógico
OR		OU lógico

Símbolos e Delimitadores

Token	Símbolo
OPEN_PARENTHESIS	(
CLOSE_PARENTHESIS	)
OPEN_BRACES	{
CLOSE_BRACES	}
OPEN_BRACKETS	[
CLOSE_BRACKETS	]
COMMA	,
SEMICOLON	;
EOF	\$

Expressões Regulares

INT → [0-9]⁺
FLOAT → [0-9]⁺.\[0-9]⁺
ID → [a-zA-Z\u0040\u00FF-9]⁺
STRING → "[a-zA-Z\u0040\u00FF_*/@=^<>!. . .]"^{*}
BOOLEAN → истинный | ложь

Exemplos Práticos

Exemplo 1: If-Else Aninhado com Operações Aritméticas

Verifica se um número está em diferentes faixas usando if-else aninhado.

Código Empress (.emp):

```
интеграл numero := 25;

если numero < 10 {
    печать "Numero esta na faixa de 0 a 9";
}
иначе {
    если numero < 20 {
        печать "Numero esta na faixa de 10 a 19";
    }
    иначе {
        если numero < 30 {
            печать "Numero esta na faixa de 20 a 29";
        }
    }
}
```

```

    }
    иначе {
        печать "Numero eh maior ou igual a 30";
    }
}
}

```

Código C gerado:

```

int numero = 25;

if (numero < 10) {
    printf("Numero esta na faixa de 0 a 9\n");
}
else {
    if (numero < 20) {
        printf("Numero esta na faixa de 10 a 19\n");
    }
    else {
        if (numero < 30) {
            printf("Numero esta na faixa de 20 a 29\n");
        }
        else {
            printf("Numero eh maior ou igual a 30\n");
        }
    }
}
}

```

Exemplo 2: Função com Operações Relacional e Aritmética

Define uma função que verifica se um número é par e calcula o dobro.

Código Empress (.emp):

```
функция verificarPar(numero) {  
    интеграл dobro := numero * 2;  
  
    если numero % 2 == 0 {  
        печать "Numero eh par";  
    }  
    иначе {  
        печать "Numero eh impar";  
    }  
  
    печать "Dobro do numero:";  
    печать dobro;  
    вернуть dobro;  
}  
  
интеграл resultado := verificarPar(7);
```

Código C gerado:

```
int verificarPar(int numero) {  
    int dobro = numero * 2;  
  
    if (numero % 2 == 0) {  
        printf("Numero eh par\n");  
    }  
    else {  
        printf("Numero eh impar\n");  
    }  
  
    printf("Dobro do numero:\n");  
    printf("%d\n", dobro);  
    return dobro;  
}  
  
int resultado = verificarPar(7);
```

Exemplo 3: While Loop com Operações Aritméticas

Calcula a soma de números usando um while loop.

Código Empress (.emp):

```
интеграл contador := 1;
интеграл soma := 0;

пока contador <= 10 {
    soma += contador;
    печать contador;
    contador++;
}

печать "Soma dos numeros de 1 a 10:";
печать soma;
```

Código C gerado:

```
int contador = 1;
int soma = 0;

while (contador <= 10) {
    soma += contador;
    printf("%d\n", contador);
    contador++;
}

printf("Soma dos numeros de 1 a 10:\n");
printf("%d\n", soma);
```


Exemplo 4: For Loop com Operações Relacional

Itera através de números com condições relacionais complexas.

Código Empress (.emp):

```
для (интеграл i := 0; i < 20; i++) {  
    если i > 5 & i < 15 {  
        печать "i esta entre 5 e 15:";  
        печать i;  
    }  
}  
  
печать "Loop finalizado";
```

Código C gerado:

```
for (int i = 0; i < 20; i++) {  
    if (i > 5 && i < 15) {  
        printf("i esta entre 5 e 15:\n");  
        printf("%d\n", i);  
    }  
}  
  
printf("Loop finalizado\n");
```

Exemplo 5: Exemplo Completo - Calculadora com Todas as Features

Exemplo que combina: função, if-else aninhado, while loop, for loop, operações aritméticas e relacionais.

Código Empress (.emp):

```
функция calcularMedia(nota1, nota2, nota3) {  
    десятичный media := (nota1 + nota2 + nota3) / 3;  
    вернуть media;  
}
```

```
функция verificarAprovacao(media) {  
    се media >= 7.0 {  
        печать "Aluno aprovado!";  
        вернуть истинный;  
    }  
    иначе {  
        если media >= 5.0 {  
            печать "Aluno em recuperacao";  
            вернуть истинный;  
        }  
        иначе {  
            печать "Aluno reprovado";  
            вернуть ложь;  
        }  
    }  
}
```

```
интеграл totalAlunos := 5;  
интеграл aprovados := 0;
```

```
для (интеграл i := 1; i <= totalAlunos; i++) {  
    печать "Digite as notas do aluno:";  
  
    desimal n1 := 8.5;  
    десятичный n2 := 7.0;  
    десятичный n3 := 9.0;  
  
    десятичный media := calcularMedia(n1, n2, n3);  
    печать "Media:";  
    печать media;  
  
    се verificarAprovacao(media) {  
        aprovados++;  
    }  
}
```

```
печать "Total de aprovados:";  
печать aprovados;
```

Precedência de Operadores

Do maior para o menor (valores mais altos vinculam mais forte):

Precedência	Operadores	Associatividade
6 (mais alto)	(), [], ++, --	Esquerda-direita
5	^ (Potência)	Direita-esquerda
4	*, /, %	Esquerda-direita
3	+, -	Esquerda-direita
2	<, <=, >, >=, ==, !=	Esquerda-direita
1 (mais baixo)	& (AND), (OR)	Esquerda-direita